

# FEN Schema for C-SQD

Facts, Episodes, and Narratives as Epistemic Audit Infrastructure

Marcus Thomas

June 2026

## Contents

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>1 Overview</b>                                | <b>3</b>  |
| 1.1 Entity Layers . . . . .                      | 3         |
| <b>2 Domain Instantiation</b>                    | <b>4</b>  |
| 2.1 Domain Configuration . . . . .               | 4         |
| 2.2 The CWE Taxonomy . . . . .                   | 5         |
| <b>3 Audit Subject</b>                           | <b>5</b>  |
| <b>4 Users and Organizations</b>                 | <b>6</b>  |
| 4.1 Reviewer Profile . . . . .                   | 7         |
| <b>5 Facts</b>                                   | <b>8</b>  |
| 5.1 FactPayload . . . . .                        | 8         |
| 5.2 FactStatus . . . . .                         | 9         |
| 5.3 Supporting Enumerations . . . . .            | 9         |
| <b>6 Audit Episodes</b>                          | <b>10</b> |
| 6.1 EpisodeMembership . . . . .                  | 11        |
| 6.2 EpisodeRelation . . . . .                    | 11        |
| <b>7 Synthesis Reviews</b>                       | <b>12</b> |
| 7.1 SynthesisSection . . . . .                   | 12        |
| 7.2 SynthesisReviewRelation . . . . .            | 13        |
| <b>8 The Evaluation Tuple</b>                    | <b>13</b> |
| 8.1 Domain-Specific Operationalization . . . . . | 14        |
| <b>9 Shared Value Types</b>                      | <b>15</b> |
| <b>10 Traits</b>                                 | <b>16</b> |
| 10.1 Temporal . . . . .                          | 16        |
| 10.2 Authored . . . . .                          | 16        |
| 10.3 FactPayload as Enum, Not Trait . . . . .    | 16        |
| 10.4 Summary of Trait Usage . . . . .            | 17        |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>11 Design Decisions</b>                                             | <b>17</b> |
| 11.1 Why the commission is a Fact, not a top-level entity . . . . .    | 17        |
| 11.2 Why AuditSubject is referenced metadata, not an Episode . . . . . | 17        |
| 11.3 Why EpisodeMembership is a standalone struct . . . . .            | 17        |
| 11.4 Why ERSolicitation and SolicitationEvent are Facts . . . . .      | 18        |
| 11.5 Why the evaluation tuple is derived, not stored . . . . .         | 18        |
| <b>12 Summary</b>                                                      | <b>18</b> |

# 1 Overview

This document specifies the internal data schema for C-SQD, adapted from the Facts, Episodes, and Narratives (FEN) architecture originally developed for Phoros. FEN organizes data into three ontological layers: atomic events (**Facts**), problem-centric groupings (**Episodes**), and authored interpretations (**Narratives**). These layers map onto C-SQD’s epistemic auditing architecture with structural fidelity.

The mapping is as follows:

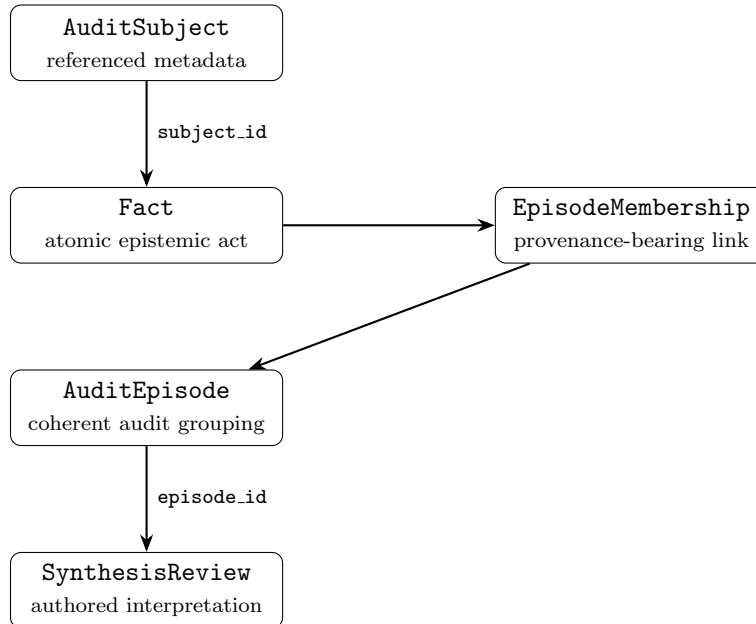
| FEN Primitive | C-SQD Realization                                           | Ontological Status                  |
|---------------|-------------------------------------------------------------|-------------------------------------|
| Fact          | <b>Fact</b> (commission, element review, solicitation, ...) | Atomic epistemic act                |
| Episode       | <b>AuditEpisode</b>                                         | Coherent audit question over time   |
| Narrative     | <b>SynthesisReview</b>                                      | Authored integrative interpretation |

The **audit subject** — a paper, model, clinical trial protocol, dataset, benchmark, or other artifact under evaluation — is not an ontological primitive. It is the **subject\_id** referent on every **Fact**: the thing the facts are *about*. This is analogous to the patient in clinical FEN. The same audit subject may be referenced by multiple independent **AuditEpisodes** commissioned by different sponsors at different times.

C-SQD extends FEN in two directions. First, **DomainInstantiation** is a first-class configuration layer above episodes, capturing the platform’s multi-domain generality: academic publishing, clinical trial review, AI auditing, policy review, and others. Second, the **evaluation tuple**  $E(A \mid R, T_{\text{eval}}) \rightarrow (N, M, S, L, U)$  is a derived view computed over accumulated **EpisodeMembership** records, recomputable for any reviewer community and reference time. It is not stored state.

The schema is defined as Rust types. Storage is abstracted.

## 1.1 Entity Layers



## 2 Domain Instantiation

C-SQD may be instantiated in multiple domains. Each instantiation defines its own CWE taxonomy, evaluation tuple configuration, and optional phase parameters. `DomainInstantiation` is a first-class entity that owns this configuration. Every downstream entity — `AuditSubject`, `Fact`, `AuditEpisode` — carries a `domain_instantiation_id` anchoring it to its instantiation.

```
struct DomainInstantiationId(Uuid);

struct DomainInstantiation {
    id:          DomainInstantiationId,
    domain_type: DomainType,
    config:       DomainConfig,
    cwe:          CWETaxonomy,
    created_at:   Timestamp,
    governed_by: Principal,
}

enum DomainType {
    AcademicPublishing,
    ClinicalTrialReview,
    AIAuditing,
    PolicyReview,
    Custom(String),
}
```

Multiple instantiations of the same `DomainType` are permitted. A sponsored NIH review program and an independent conference partnership are both `AcademicPublishing` instantiations with different configurations.

### 2.1 Domain Configuration

```
struct DomainConfig {
    eval_tuple_config: EvalTupleConfig,
    audit_subject_types: Vec<AuditSubjectType>,
    phase_config:       Option<PhaseConfig>,
}
```

`PhaseConfig` is optional. Commissioned audits do not require the three-phase public review structure; it is available for domain instantiations that need it.

```
struct PhaseConfig {
    public_review_duration: Option<Duration>,
    response_rounds_permitted: u32,
    synthesis_significance_threshold: f64,
    anonymity_rules:           AnonymityConfig,
}
```

## 2.2 The CWE Taxonomy

The Common Weakness Enumeration (CWE) is an extensible node tree of evaluation criteria. It is not a closed enum. Each node carries provenance indicating whether it belongs to the base taxonomy or a community extension.

```
struct CWENodeId(Uuid);

struct CWENode {
    id: CWENodeId,
    domain_instantiation_id: DomainInstantiationId,
    parent: Option<CWENodeId>,
    label: String,
    description: String,
    source: CWESource,
}

enum CWESource {
    BaseTaxonomy,
    CommunityExtension { community_id: CommunityId }, // deferred: post-MVP
}

// A CWECriterionId carries its domain context implicitly.
struct CWECriterionId {
    domain: DomainInstantiationId,
    node_id: CWENodeId,
}
```

## 3 Audit Subject

An `AuditSubject` is the artifact under evaluation: a research manuscript, preprint, dataset, code repository, clinical trial protocol, AI model evaluation, benchmark, policy document, grant proposal, or technical report. It is *not* an ontological primitive in the FEN sense. It is the referent of `subject_id` on every `Fact` — the thing the facts are about. The design rationale is discussed in Section 11.2.

```
struct AuditSubjectId(Uuid);

struct AuditSubject {
    id: AuditSubjectId,
    domain_instantiation_id: DomainInstantiationId,
    subject_type: AuditSubjectType,
    title: Option<String>,
    external_refs: Vec<ExternalRef>,
    registered_by: Principal,
    registered_at: Timestamp,
}

enum AuditSubjectType {
    ResearchManuscript,
    Preprint,
}
```

```

    Dataset,
    CodeRepository,
    ClinicalTrialProtocol,
    AIModelEvaluation,
    Benchmark,
    PolicyDocument,
    GrantProposal,
    TechnicalReport,
    Other(String),
}

```

An `AuditSubject` may be registered by a sponsor, an admin, or an importer. It does not need to have been submitted by its author. The same subject may be referenced by multiple independent `AuditEpisodes`.

## 4 Users and Organizations

Every platform participant is a `User`. Reviewer and submitter are roles that context supplies, not distinct types. A `ReviewEvent` records who reviewed; an `AuditCommission` fact records who sponsored. A user who has completed reviewer onboarding carries a `ReviewerProfile`.

```

struct UserId(Uuid);

struct User {
    id:          UserId,
    display_name: String,
    email:       String,
    created_at:  Timestamp,
    status:      UserStatus,
    reviewer_profile: Option<ReviewerProfile>,
}

enum UserStatus {
    Active,
    Suspended { reason: String, until: Option<Timestamp> },
    Deactivated,
}

```

`Organization` is a first-class entity representing institutional sponsors: biotechnology companies, venture capital firms, foundations, universities, journals, and regulators.

```

struct OrganizationId(Uuid);

struct Organization {
    id:          OrganizationId,
    name:        String,
    org_type:    OrganizationType,
    created_at:  Timestamp,
}

enum OrganizationType {
    Biotech,
}

```

```

    VentureCapital,
    Foundation,
    University,
    Journal,
    Regulator,
    Other(String),
}

```

`Principal` is the reference type used in provenance and authorship contexts. It identifies who asserted or performed something without carrying the full entity record.

```

enum Principal {
    User(UserId),
    Organization(OrganizationId),
    Platform,
    AiAssisted { tool_id: String, supervised_by: Option<UserId> },
}

```

## 4.1 Reviewer Profile

`ReviewerProfile` is a top-level entity keyed by `UserId`, fetched independently when reviewer-specific context is needed.

```

struct ReviewerProfile {
    user_id:      UserId,
    status:       ReviewerStatus,
    tags:         Vec<ReviewerTag>,
    domain_extensions: Vec<ReviewerDomainExtension>,
}

enum ReviewerStatus {
    GracePeriod,
    Active,
    Suspended,
}

struct ReviewerTag {
    id:      TagId,
    label:   String,
    scope:   TagScope,
    verified: bool,
}

enum TagScope {
    Global,
    Domain(DomainInstantiationId),
}

struct ReviewerDomainExtension {
    domain_instantiation_id: DomainInstantiationId,
    data: serde_json::Value, // domain-specific fields
}

```

## 5 Facts

A **Fact** is the atomic unit of the C-SQD schema. It represents a single epistemic or administrative act grounded in provenance: a sponsored audit commission, a focused element review, a solicitation issued to a reviewer, a solicitation lifecycle event, or a submitter response. Facts are time-indexed, provenance-rich, and immutable. They are not updated; if a Fact is wrong or superseded, its **status** changes and a new Fact may be created.

```
struct FactId(Uuid);

struct Fact {
    id: FactId,
    subject_id: AuditSubjectId,
    domain_instantiation_id: DomainInstantiationId,
    occurred_at: Timestamp,
    payload: FactPayload,
    status: FactStatus,
    provenance: Provenance,
    external_refs: Vec<ExternalRef>,
}
```

### 5.1 FactPayload

**FactPayload** is a sum type carrying the type-specific data for each fact variant. The set of variants is closed and controlled by C-SQD. This design gives exhaustive pattern matching, value semantics, and no heap allocation. The rationale for using an enum rather than trait objects is discussed in Section 10.3.

```
enum FactPayload {
    AuditCommission {
        commissioned_by: Principal,
        scope: Vec<CWECriterionId>,
        funding: Money,
        deadline: Option<Timestamp>,
        confidential: bool,
    },
    ElementReview {
        cwe_criterion: CWECriterionId,
        submitted_by: UserId,
        solicitation: Option<FactId>, // ref to ERSolicitation fact
        finding: Finding,
        severity: Option<FindingSeverity>,
        confidence: Option<ConfidenceLevel>,
        limitations: Option<String>,
        recommendations: Option<String>,
        content: String,
        featured: bool,
    },
    ERSolicitation {
        issued_to: UserId,
        cwe_criterion: CWECriterionId,
        commission: FactId, // ref to AuditCommission fact
    },
}
```

```

        payment_scheme: PaymentScheme,
    },
    SolicitationEvent {
        solicitation: FactId,          // ref to ERSolicitation fact
        event_type: SolicitationEventType,
        principal: Principal,
        note: Option<String>,
    },
    SubmitterResponse {
        responding_to: Vec<FactId>,
        response_type: ResponseType,
        content: String,
        revision_ref: Option<AuditSubjectId>,
    },
}

```

ERSolicitation and SolicitationEvent are Fact variants because they are atomic, timestamped institutional acts. A solicitation is an act of assignment; a solicitation event is an act of acceptance, completion, or declination. References between fact variants use **FactId**; the application layer verifies that the referenced Fact carries the expected payload variant. The rationale is discussed in Section 11.4.

## 5.2 FactStatus

Facts are not versioned. If a fact is wrong or superseded, its status is set and a new fact may be created to replace it.

```

enum FactStatus {
    Active,
    Superseded {
        superseded_by: Principal,
        superseded_at: Timestamp,
        replaced_by: Option<FactId>,
    },
    Retracted {
        retracted_by: Principal,
        retracted_at: Timestamp,
        reason: Option<String>,
    },
}

```

## 5.3 Supporting Enumerations

```

enum Finding {
    NonEthicalProblem,
    EthicalProblem,
    NoProblems,
    Inconclusive,
}

enum FindingSeverity { Minor, Moderate, Major, Critical }

```

```

enum ConfidenceLevel { Low, Moderate, High }

enum SolicitationEventType {
    Issued,
    Accepted,
    Declined,
    Expired,
    Completed,
}

enum ResponseType {
    Accepts,
    Contests,
    PartiallyAccepts,
    RevisesAndResponds,
}

```

## 6 Audit Episodes

An **AuditEpisode** is the persistent object representing a coherent audit question over time. It is not a time window. It is the assertion that a set of **Facts** belong to the same audit. Multiple episodes may reference the same **AuditSubject**: different sponsors commissioning independent audits of the same artifact produce separate, fully independent episodes.

```

struct AuditEpisodeId(Uuid);

struct AuditEpisode {
    id: AuditEpisodeId,
    subject_id: AuditSubjectId,
    domain_instantiation_id: DomainInstantiationId,
    label: String,
    status: EpisodeStatus,
    authored_by: Principal,
    authored_at: Timestamp,
    notes: Option<String>,
}

enum EpisodeStatus {
    Active,
    SynthesisPending,
    Delivered,
    Closed,
}

```

An episode is opened when an **AuditCommission** fact is created and linked to it via **EpisodeMembership** with role **Commission**. Phase state (where applicable) is derived from **PhaseConfig** parameters and the commission fact's **occurred\_at**; it is not stored as mutable state on the episode.

## 6.1 EpisodeMembership

EpisodeMembership links a Fact to an AuditEpisode. It is a standalone struct, not a list on AuditEpisode. The episode is defined by the set of memberships that point to it, not by an internal collection it owns. The rationale is discussed in Section 11.3.

```
struct MembershipId(Uuid);

struct EpisodeMembership {
    id:          MembershipId,
    fact_id:     FactId,
    episode_id:  AuditEpisodeId,
    role:        FactRole,
    asserted_by: Principal,
    asserted_at: Timestamp,
    status:      MembershipStatus,
}

enum FactRole {
    Commission,
    ElementReview,
    Solicitation,
    SolicitationLifecycle,
    Response,
    Administrative,
    Other,
}

enum MembershipStatus {
    Active,
    Retracted {
        retracted_by: Principal,
        retracted_at: Timestamp,
    },
}
```

## 6.2 EpisodeRelation

EpisodeRelation represents an asserted relationship between two episodes. These relationships are not inferred; they are authored claims with provenance and can be audited, contested, or removed.

```
struct EpisodeRelation {
    id:          RelationId,
    source_episode_id: AuditEpisodeId,
    target_episode_id: AuditEpisodeId,
    relation_type: EpisodeRelationType,
    asserted_by:  Principal,
    asserted_at:  Timestamp,
}

enum EpisodeRelationType {
    Supersedes,
```

```
    SplitFrom,  
    MergedInto,  
    RelatedTo,  
    PartOf,  
}
```

**Semantics of PartOf.** `PartOf` models structural containment: a lower-level episode is one component of a broader higher-level episode. The direction is component  $\rightarrow$  aggregate. For example, a multi-domain audit of the same artifact may be represented as a higher-level episode composed of several domain-specific sub-episodes, each with independent authorship, timestamps, and contestability.

## 7 Synthesis Reviews

A `SynthesisReview` is an authored integrative interpretation of an `AuditEpisode`. It is the C-SQD realization of the FEN Narrative primitive. Multiple synthesis reviews per episode is the normal case; reviewers may agree, partially overlap, or contest each other.

```
struct SynthesisReviewId(Uuid);  
  
struct SynthesisReview {  
    id:          SynthesisReviewId,  
    episode_id:  AuditEpisodeId,  
    submitted_by: UserId,  
    authored_at: Timestamp,  
    status:      NarrativeStatus,  
    summary:     String,  
    sections:    Vec<SynthesisSection>,  
    featured:    bool,  
}  
  
enum NarrativeStatus {  
    Draft,  
    Current,  
    Superseded,  
}
```

### 7.1 SynthesisSection

`SynthesisSection` provides optional internal structure within a synthesis review. Not all reviews require sections; the top-level `summary` field may suffice for straightforward audits. Sections are useful when the review distinguishes evaluative dimensions or references specific facts within a larger argument.

```
struct SectionId(Uuid);  
  
struct SynthesisSection {  
    id:          SectionId,  
    review_id:   SynthesisReviewId,
```

```

    section_type:      SynthesisSectionType,
    content:           String,
    referenced_facts:  Vec<FactId>,
}

enum SynthesisSectionType {
    Summary,
    MethodologicalAssessment,
    EthicalAssessment,
    EvidenceIntegration,
    Recommendations,
    OpenQuestions,
}

```

## 7.2 SynthesisReviewRelation

Multiple synthesis reviews per episode may agree, partially overlap, or contest each other. `SynthesisReviewRelation` records these relationships explicitly as authored claims with provenance.

```

struct SynthesisReviewRelation {
    id:           RelationId,
    source:       SynthesisReviewId,
    target:       SynthesisReviewId,
    relation_type: NarrativeRelationType,
    asserted_by:  Principal,
    asserted_at:  Timestamp,
}

enum NarrativeRelationType {
    Supersedes,
    Contests(ContestationInfo),
    RelatedTo,
}

struct ContestationInfo {
    scope:        ContestationScope,
    rationale:    Option<String>,
}

enum ContestationScope { Partial, Full }

```

## 8 The Evaluation Tuple

The evaluation tuple is a computed view, not stored state. It is a pure function over accumulated `EpisodeMembership` records, filtered by reviewer community and reference time. Its purity over immutable inputs makes it deterministic and suitable for future client-side recomputation.

$$E(A \mid R, T_{\text{eval}}) \rightarrow (N, M, S, L, U)$$

where  $A$  is an `AuditEpisode`,  $R$  is a reviewer community filter, and  $T_{\text{eval}}$  is a reference time. The five components:

- $N$ : non-ethical problems identified by reviewers in  $R$  up to  $T_{\text{eval}}$
- $M$ : ethical problems identified by reviewers in  $R$  up to  $T_{\text{eval}}$
- $S$ : stakes of the audit subject, operationalized per domain instantiation
- $L$ : scrutiny depth, weighting `ElementReview` facts by reviewer expertise and solicitation status
- $U$ : uptake of the audit subject and its audit record, operationalized per domain instantiation

```
fn compute_eval_tuple(
    episode_id: AuditEpisodeId,
    community: ReviewerCommunityFilter,
    t_eval:    Timestamp,
    config:    &EvalTupleConfig,
) -> EvalTuple

struct EvalTuple {
    n:          f64,
    m:          f64,
    s:          f64,
    l:          f64,
    u:          f64,
    computed_at: Timestamp,
    community_filter: ReviewerCommunityFilter,
}

struct ReviewerCommunityFilter {
    tags:          Vec<TagId>,
    domain_instantiation_id: Option<DomainInstantiationId>,
    min_endorsements: Option<u32>,
}
```

## 8.1 Domain-Specific Operationalization

$S$  and  $U$  vary in meaning across domain instantiations. `EvalTupleConfig` on `DomainConfig` captures this variation.

```
struct EvalTupleConfig {
    stakes_operationalization: StakesDefinition,
    uptake_operationalization: UptakeDefinition,
    l_weight_params:          ScrutinyWeightParams,
}

enum StakesDefinition {
    ScientificSignificance,
    PublicHealthConsequentiality,
    DeploymentRiskProfile,
}
```

```

    Custom(String),
}

enum UptakeDefinition {
    CitationImpact,
    DownstreamAdoption,
    DeploymentDecisions,
    Custom(String),
}

struct ScrutinyWeightParams {
    solicited_review_multiplier: f64,
    expertise_weight_fn:      String, // identifier for weighting function
}

```

## 9 Shared Value Types

```

struct Provenance {
    source_system: Option<String>,
    source_document: Option<DocumentId>,
    imported_at: Timestamp,
    principal: Principal,
}

struct PaymentScheme {
    amount: Money,
    currency: String,
    condition: PaymentCondition,
}

enum PaymentCondition {
    OnSubmission,
    OnAcceptance,
    Tiered { on_submission: Money, on_acceptance: Money },
}

struct Money {
    amount: f64,
    currency: String,
}

struct ExternalRef {
    system: ExternalSystem,
    resource_type: Option<String>,
    resource_id: String,
    uri: Option<String>,
}

enum ExternalSystem {
    DOI,
    ArXiv,
}

```

```
ClinicalTrialsGov,  
ORCID,  
GitHub,  
HuggingFace,  
Other(String),  
}
```

## 10 Traits

The schema uses traits where the system needs polymorphic dispatch over shared behavior. Two traits are warranted.

### 10.1 Temporal

The system must place heterogeneous objects on a single timeline. **Fact**, **EpisodeMembership**, **EpisodeRelation**, and **SynthesisReview** all implement this trait, enabling timeline views that interleave entity types without matching on every concrete type.

```
trait Temporal {  
    fn temporal_anchor(&self) -> Timestamp;  
}
```

### 10.2 Authored

The system must answer provenance queries across entity types — audit logs, sponsor-filtered views, and conflict disclosure all require knowing who asserted what and when across entity boundaries.

```
trait Authored {  
    fn principal(&self) -> &Principal;  
    fn authored_at(&self) -> Timestamp;  
}
```

### 10.3 FactPayload as Enum, Not Trait

The **FactPayload** variant set is closed and controlled by C-SQD. A trait hierarchy of separate fact structs would be wrong here. A closed set of variants is an enum, not a trait hierarchy. Enums provide exhaustive pattern matching: the compiler enforces that every handler covers every variant. They require no heap allocation and allow value semantics. Trait objects lose all three properties.

If C-SQD's fact types were an open set — if external systems could register new fact types at runtime — trait objects would be justified. That is not the current design. Adding a new fact type means adding a variant to **FactPayload**; the compiler then identifies every **match** expression that needs updating.

## 10.4 Summary of Trait Usage

| Pattern                 | Mechanism         | Reason                                   |
|-------------------------|-------------------|------------------------------------------|
| Timeline placement      | Temporal trait    | Heterogeneous collection, single sort    |
| Provenance queries      | Authored trait    | Generic “who authored this” across types |
| Fact-type-specific data | FactPayload enum  | Closed set, exhaustive matching          |
| Shared fields only      | Repeat the fields | No behavioral need for abstraction       |

## 11 Design Decisions

### 11.1 Why the commission is a Fact, not a top-level entity

A common instinct is to model the audit commission as a first-class entity alongside `AuditEpisode`. This is wrong for the same reason that billing and insurance events in clinical FEN are `Fact` variants rather than separate entities: a commission is an institutional act. It happens at a point in time, it is authored, it is provenance-bearing, and it is immutable. These are the properties of a `Fact`.

Modeling the commission as a `FactPayload` variant also keeps the entity model minimal. The `AuditEpisode` is opened when an `AuditCommission` fact is created and linked to it via `EpisodeMembership` with role `Commission`. The commission fact carries the funding, scope, and sponsor identity. The episode carries the grouping logic.

### 11.2 Why AuditSubject is referenced metadata, not an Episode

The audit subject — the paper, model, or trial — is the thing the facts are *about*. It is not itself a fact, episode, or narrative. In clinical FEN, the patient occupies the same structural position: every `Fact` carries a `subject_id`, but the patient is not in the FEN ontology.

This separation has a concrete consequence: the same paper can be the subject of multiple independent `AuditEpisodes` commissioned by different sponsors at different times. Each episode has its own facts, its own memberships, and its own synthesis reviews. They do not collapse into a single entity merely because they share a subject. A venture capital firm evaluating an academic finding for a licensing decision and an independent journal seeking external review of the same paper produce entirely separate audit records.

### 11.3 Why EpisodeMembership is a standalone struct

An `AuditEpisode` does not contain a `Vec<FactId>`. Instead, `EpisodeMembership` is a standalone struct that references both a `FactId` and an `AuditEpisodeId`. The episode is defined by the set of memberships that point to it, not by an internal collection it owns.

This design has four advantages. First, **granular contestability**: assigning a fact to an episode is itself a claim and needs provenance. Two principals may independently assert that the same element review belongs to different episodes. Both memberships exist simultaneously; the disagreement lives in the membership records. Second, **multi-episode membership**: a single fact can belong to multiple episodes without duplication. Third, **audit history**: the full history of grouping decisions is preserved; if a fact is reassigned, the old membership is retracted and a new one created. Fourth, **role annotation**: the same fact may serve different roles (`ElementReview`, `Administrative`) in different episodes.

## 11.4 Why ERSolicitation and SolicitationEvent are Facts

A solicitation (the platform issuing a reviewer assignment) and a solicitation lifecycle event (the reviewer accepting, completing, or declining) are atomic institutional acts. They belong in **FactPayload** for the same reason billing and pre-authorization events belong there in clinical FEN: each is time-stamped, provenance-bearing, and immutable.

Modeling them as separate top-level entities would fragment the provenance model without structural benefit. As **Fact** variants they are linked to the **AuditEpisode** via **EpisodeMembership**, appear on the episode timeline via the **Temporal** trait, and participate in the same audit history as the element reviews they coordinate.

References between solicitation facts use **FactId**. The application layer verifies that the referenced **Fact** carries the expected payload variant.

## 11.5 Why the evaluation tuple is derived, not stored

The evaluation tuple is a pure function over immutable **EpisodeMembership** records. Storing it would introduce redundancy and stale-state risk without benefit, since the inputs are themselves immutable and the computation is deterministic for any fixed  $(A, R, T_{\text{eval}})$  triple.

Different consumers may legitimately compute the tuple with different community filters. A drug regulator recomputes using only clinical pharmacologist reviewers. A policy analyst weights  $S$  and  $U$  differently. A safety team filters by reviewers with demonstrated expertise in a specific CWE criterion. Storing a single tuple would collapse this flexibility into a single institutional perspective.

## 12 Summary

The C-SQD schema defines entity types across five layers:

| Layer         | Entities                                                                          | Ontological Status                             |
|---------------|-----------------------------------------------------------------------------------|------------------------------------------------|
| Configuration | <b>DomainInstantiation</b> , <b>DomainConfig</b> , <b>CWENode</b>                 | Domain configuration                           |
| Subject       | <b>AuditSubject</b>                                                               | Referenced artifact metadata                   |
| Users         | <b>User</b> , <b>Organization</b> , <b>ReviewerProfile</b>                        | Identity and reviewer status                   |
| Facts         | <b>Fact</b> (with <b>FactPayload</b> )                                            | Atomic epistemic acts                          |
| Episodes      | <b>AuditEpisode</b> , <b>EpisodeMembership</b> , <b>EpisodeRelation</b>           | Coherent audit groupings                       |
| Narratives    | <b>SynthesisReview</b> , <b>SynthesisSection</b> , <b>SynthesisReviewRelation</b> | Authored interpretations                       |
|               |                                                                                   |                                                |
|               | <b>Trait</b>                                                                      | <b>Purpose</b>                                 |
|               | <b>Temporal</b>                                                                   | Unified timeline placement across entity types |
|               | <b>Authored</b>                                                                   | Cross-entity provenance queries                |

The evaluation tuple  $E(A \mid R, T_{\text{eval}}) \rightarrow (N, M, S, L, U)$  is a derived view, not stored state. It is a pure function over immutable **EpisodeMembership** records, recomputable for any reviewer community and reference time.

The three central structural decisions, inherited from FEN: **Fact-to-AuditEpisode** assignment is a standalone provenance-bearing record rather than a list on the episode; **AuditSubject** is

referenced metadata rather than an ontological primitive; and the commission that opens an episode is a **FactPayload** variant rather than a separate top-level entity.

The two central extensions beyond FEN: **DomainInstantiation** is a first-class entity that owns the CWE taxonomy and evaluation tuple configuration, making C-SQD's domain generality explicit in the type system; and the evaluation tuple introduces a derived multi-dimensional assessment view that has no clinical FEN equivalent.